

Base + Step $\Rightarrow$ Forever

One inductive invariant in three costumes: a crypto proof, a safety certificate, and the recursor of Lean

A. Author¹

Somewhere University, a.author@example.edu

Abstract. A property that is *true at the start* and *kept by every step* is *true forever*. That single shape — an *inductive invariant* — wears three costumes. It is a *conserved quantity* inside a cryptography proof (Dobbertin 1999, Theorem 6, eq. (11)); it is a *safety certificate* in model checking (mutual exclusion); and it is the *recursor* of an inductive type in Lean 4 / CIC. We state the pattern once, instantiate it three times, and show the last costume is literal: the soundness theorem *is* `Reachable.rec`, checked by `rf1`, using no axioms. The whole thing builds with no `sorry`. This is a pearl of *recognition*: one idea, spotted in a real proof, and followed to the foundation.

Keywords: verification pearl · inductive invariant · model checking · characteristic 2 · Lean 4 · CIC

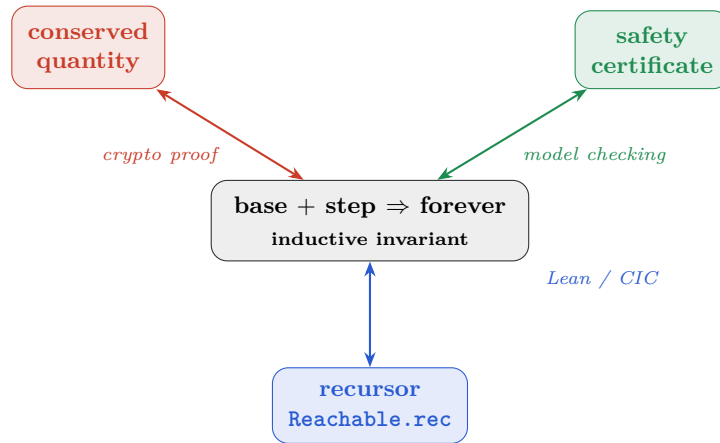


Fig. 1. The pearl in one picture. The same invariant wears three costumes. Each double arrow reads “same shape, different words”. This paper draws each costume and lets Lean check that they are the one shape.

1 The pattern, stated once

A *transition system* is a set of start states and a step relation. *Reachable* is what you can get to. An *inductive invariant* P holds at every start state and survives every step. The one load-bearing fact: such a P then holds at *every* reachable state. That is the whole method — P is the *certificate*, the theorem is the *check*.

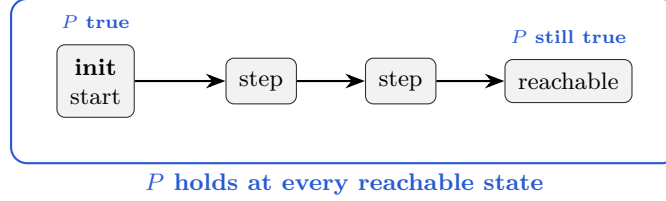


Fig. 2. P true at the start + every step keeps $P \Rightarrow P$ true everywhere. The blue box is the theorem below.

```

structure InductiveInvariant (T : TransitionSystem State) (P : State -> Prop) : Prop where
  init_holds      : forall s, T.init s -> P s
  step_preserves  : forall s s', P s -> T.step s s' -> P s'

theorem InductiveInvariant.reachable (h : InductiveInvariant T P) :
  forall s, Reachable T s -> P s

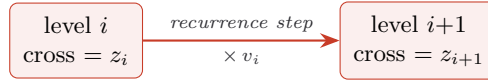
```

Everything below is one instance of `InductiveInvariant.reachable`.

2 Costume 1 — a conserved quantity in a crypto proof

The pattern was *spotted*, not invented, inside the proof of Dobbertin’s Theorem 6 over $\mathbb{F}_{2^n}$. Two polynomial sequences A, B obey the same two-step recurrence $s_{i+2} = u_i s_{i+1} + v_i s_i$. Their bilinear *cross term* is conserved (eq. (11)):

$$A_i B_{i+1} + A_{i+1} B_i = z_i, \quad z_{i+1} = v_i z_i.$$



char. 2: the two diagonal terms $u_i A_{i+1} B_{i+1}$ are equal, so they cancel. That cancellation is why the cross term is conserved.

Fig. 3. Costume 1. The cross term is the invariant; one recurrence step multiplies it by v_i . “Holds at $i = 0$, preserved by each step, therefore holds for all i ” is exactly base + step $\Rightarrow$ forever.

```

theorem cross_invariant (h : TwoStep u v A B) (i : N) :
  A (i+1) * B (i+2) + A (i+2) * B (i+1) = v i * (A i * B (i+1) + A (i+1) * B i)

theorem key_identity (h : TwoStep u v A B) (hz : forall i, z (i+1) = v i * z i)
  (h0 : A 0 * B 1 + A 1 * B 0 = z 0) (i : N) :
  A i * B (i+1) + A (i+1) * B i = z i      -- eq. (11), by induction on i

```

`key_identity` is a hand-rolled induction: base (`h0`), step (`cross_invariant`), done. The pattern is there — but disguised as algebra.

3 Costume 2 — a safety certificate in model checking

Give the shape its own vocabulary and it becomes the textbook safety recipe. Two processes share one lock; safety is: never both in the critical section.

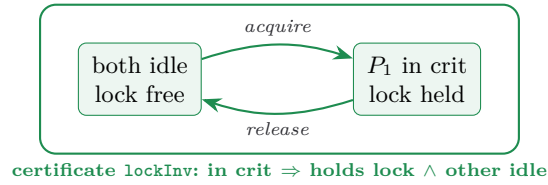


Fig. 4. Costume 2. The invariant `lockInv` is the certificate; feeding it to `safety_via_invariant` discharges mutual exclusion. Same shape as Fig. 3, familiar example.

```

theorem mutualExclusion_reachable (s : St) (hs : Reachable mutexSystem s) :
  mutualExclusion s :=
  safety_via_invariant lockInv_inductive lockInv_imp_mutualExclusion s hs

```

4 Costume 3 — the recursor of an inductive type

Why is `InductiveInvariant.reachable true`? Because `Reachable` is an ordinary Lean inductive: its two constructors are exactly the two rules “start” and “step”. Soundness is nothing more than its auto-generated recursor, fed the invariant’s two fields.

```

inductive Reachable (S : System sigma) : sigma -> Prop
| start : S.init s -> Reachable S s      -- rule 1 (= init_holds)
| step  : Reachable S s -> S.step s s' -> Reachable S s' -- rule 2 (= step_pres)

```

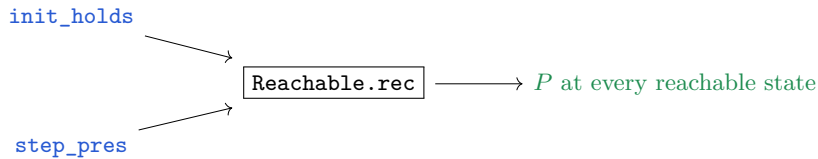


Fig. 5. Costume 3, as a commuting picture. The two invariant fields are the two arguments the recursor `Reachable.rec` takes; its output is soundness. Nothing else is needed.

```

-- soundness, written by hand-applying the recursor:
theorem Inductive.reachable' (h : Inductive S P) : forall s, Reachable S s -> P s :=
  fun _ hs => Reachable.rec (fun hi => h.init_holds _ hi)
    (fun _ hstep ih => h.step_pres _ _ ih hstep) hs

-- the "by induction" proof and the raw-recursor proof are the SAME term:
example (h : Inductive S P) : Inductive.reachable h = Inductive.reachable' h := rfl

```

The `rfl` is the pearl's punchline: `by induction` elaborates to `Reachable.rec`.
 Model-checking soundness is not merely *proved by* induction — it *is* CIC
 induction. `InductiveInvariant.reachable` depends on **no axioms**.

5 Closing the loop — the crypto identity, re-derived

The circle closes when we re-tell Costume 1 in the vocabulary of Costume 2. Make the state a sliding window $(i, A_i, A_{i+1}, B_i, B_{i+1})$ and the step the recurrence. Then the cross term is an inductive invariant, and eq. (11) drops out of the *generic* soundness theorem — a faithful “morphism” from the crypto proof to the model-checking core.

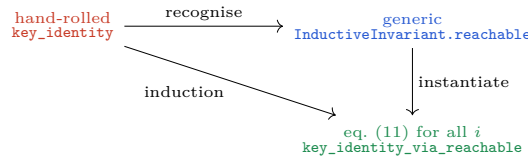


Fig. 6. The loop commutes. The identity spotted by hand (top-left) and the same identity produced by the generic machinery (bottom) agree; recognising the pattern is the diagonal that makes them one.

```

theorem cross_is_inductiveInvariant (hz : forall i, z (i+1) = v i * z i) :
  InductiveInvariant (recSystem u v z) (crossInv z) -- cross term is an invariant

theorem key_identity_via_reachable ... (i : N) :
  A i * B (i+1) + A (i+1) * B i = z i -- eq. (11), now an instance
:= cross_holds_of_reachable ... (windowState_reachable ... i)

```

6 Three costumes, one table

Table 1. The one shape, read three ways. Every row is a machine-checked theorem.

	crypto	model checking	Lean / CIC
state	level i , window A, B	lock + two locations	a term of Reachable
invariant	cross term $= z_i$	lockInv	the motive P
base	eq. (11) at $i = 0$	both idle, lock free	init_holds
step	char. 2 cancellation	acquire / release	step_pres
conclusion	eq. (11) for all i	never both in crit	P everywhere reachable
theorem	key_identity	mutualExclusion_	InductiveInvariant.
	_via_reachable	reachable	reachable = .rec

7 What transfers, and what this is not

What transfers. Whenever you can name a quantity that (i) holds at the start and (ii) is preserved by one step, you already have a proof for the whole run — and, in a proof assistant, that proof is free: it is the recursor. The skill is *recognition*, not machinery.

Honest scope. This is the soundness *kernel* certifying model checking rests on — done by hand in a proof assistant. It is not an automated or SMT-backed checker and does not explore millions of states. Real tools scale this to large systems; here it is deliberately small, so the core is fully understood and fully checked.

Artifact. Lean 4 / Mathlib (v4.28.0). Builds with `lake build, no sorry`. Anchors: `InductiveInvariant.reachable` (no axioms); `Inductive.reachable = Inductive.reachable'` by `rfl`; `key_identity_via_reachable` re-derives eq. (11). Six short files, one per section.

Table 2. Mindful design log (Ellen Langer style): a usual “default” of pearl writing, and the option taken here once it is noticed as *just* a default.

usual default	option taken here
front-load prose motivation	open with Fig. 1; captions motivate
write proofs in text	proofs as commuting diagrams (Figs. 5,6)
a new result	re-derivation-as-pearl: recognise a known shape
figures illustrate the text	text annotates the figures
related work is a section	positioned in one table (Table 1)
contributions as a bullet list	one overview figure, three costumes
fill the page limit	brevity is the contribution

Acknowledgement. The Lean development was carried out with machine assistance; all statements are checked by the Lean kernel.